mini-DM — instrukcja wdrożenia na serwer demo

Wersja aplikacji	commit `fb87c13` (gałąź `main`)
Poprzednio wdrożona wersja	commit `69b80f5` (wdrożenie z 2026-11-08)
Data przygotowania	2026-11-13
Środowisko	serwer demo klienta (`<ADRES_DEMO>`), Docker + docker compose, reverse proxy nginx

0. Konto i uprawnienia

Całe wdrożenie wykonuj ze zwykłego konta administratora serwera, `<UZYTKOWNIK>` (zob. punkt 2): przesyłanie plików z Windows przez WinSCP, logowanie przez SSH i polecenia na serwerze. **Nie loguj się jako root.** Wtedy pomyłka w poleceniu `rm` lub `unzip` w złym katalogu nie ma pełnych uprawnień.

- `<UZYTKOWNIK>` musi należeć do grupy `docker` (sprawdź poleceniem `id`). Wtedy `docker build`, `docker tag`, `docker run` i `docker compose` działają **bez `sudo`**, co potwierdzi polecenie `docker ps`. Jeśli zgłasza `permission denied`, wyloguj się i zaloguj ponownie, bo członkostwo w grupie działa dopiero w nowej sesji.
- Konto, z którego uruchamiasz `docker build`, nie ma wpływu na obraz. Obraz i tak buduje demon Dockera, a właściciela plików w obrazie ustala `Dockerfile` (użytkownik `app`).
- `<KATALOG_ZRODEL>` i `<KATALOG_KOPII>` powinny należeć do `<UZYTKOWNIK>`, np. `~/mini-dm-src` i `~/kopie-mini-dm`.
- **`sudo` jest potrzebne tylko** przy operacjach na plikach bazy w `<KATALOG_DANYCH>`, jeśli nie należą one do `<UZYTKOWNIK>`. Dotyczy to `chmod` w kroku 6 oraz przywracania bazy w kroku 9. W tych miejscach polecenia mają już `sudo`.

1. Co zawiera to wdrożenie

Zmiany względem `69b80f5`:

1. **Nowe pole „Armator”** (przewoźnik morski), tekst do 50 znaków, niewymagane. Jest tylko na formularzu **edycji** zamówienia, obok pola „Magazyn”. Formularz dodawania zamówienia się nie zmienia. Zmiana armatora trafia do historii zmian zamówienia.
2. **Lista zamówień: kolumna „Cena za kg” zamiast „Wartość”.** Pokazuje cenę portową za kilogram z walutą zamówienia. Wartość zamówienia nadal jest w formularzu edycji, a raport nadal sumuje wartości zamówień.

To wdrożenie zmienia strukturę bazy. Zawiera **migrację nr 5, która przebudowuje tabelę `purchase_orders`**: nowa kolumna `shipping_line` staje między `warehouse_name` a `has_eur1_certificate`. Istniejące zamówienia dostają puste pole „Armator”, a pozostałe dane, historia zmian i indeksy zostają zachowane. **Kopia zapasowa bazy (krok 4) jest obowiązkowa.**

Nie zmieniają się zmienne środowiskowe, port (4321) ani konfiguracja `docker-compose.yml` i nginx.

> ***Jeśli serwer ma wersję starszą niż `69b80f5`**, to wdrożenie zawiera też zmiany z wcześniejszych wdrożeń: dymek z powodem oznaczenia „wymaga uwagi” (`69b80f5`) i filtr oraz kafelek „Brak badań dostawcy” (`d1b958a`). Jeśli jest starsza niż `95062ff` (sprzed migracji nr 4), zawiera ponadto:*
>
> - *pole „Badania laboratoryjne od dostawcy”,*

> - *przeniesione pole „Zablokowane",*
> - *poszerzony layout,*
> - ***migrację nr 4, która również przebudowuje tabelę*** `purchase_orders` *(wykona się przed migracją nr 5).*
>
> *Różnice w oczekiwanych wynikach opisują uwagi „_starsza wersja_" w krokach 4.4, 8 i 9.*

2. Oznaczenia używane w instrukcji

Symbol	Znaczenie
`<UZYTKOWNIK>`	zwykłe (nie-root) konto na serwerze, w grupie `docker`, używane też w WinSCP
`<ADRES_DEMO>`	domena, pod którą działa serwer demo
`<KATALOG_ZRODEL>`	katalog na serwerze, w którym budowany jest obraz (kopia repozytorium)
`<KATALOG_COMPOSE>`	katalog z plikiem `docker-compose.yml`, w którym zdefiniowana jest usługa `mini-dm`
`<KATALOG_DANYCH>`	katalog z bazą SQLite, podmontowany w kontenerze jako `/app/data` (bind mount z `docker-compose.yml`)
`<KATALOG_KOPII>`	dowolny katalog na kopie zapasowe, poza `<KATALOG_DANYCH>`

Rzeczywiste wartości tych oznaczeń celowo nie są zapisane w repozytorium. Ścieżki odczytasz z `docker-compose.yml` na serwerze.

Instrukcja zakłada, że usługa `mini-dm` w `docker-compose.yml` korzysta z obrazu `mini-dm:latest`, tak jak przy poprzednich wdrożeniach.

3. Przygotowanie paczki ze źródłami (lokalnie)

Na komputerze deweloperskim, w katalogu projektu:

```
git archive --format=zip -o mini-dm-fb87c13.zip fb87c13
```

`git archive` pakuje tylko pliki śledzone przez git. W paczce nie ma więc `node_modules`, `dist`, `.astro`, `data`, `.env*` ani lokalnych konfiguracji.

Skopiuj `mini-dm-fb87c13.zip` na serwer przez WinSCP, logując się jako `<UZYTKOWNIK>`, do katalogu domowego (`~`). Następnie wyczyść `<KATALOG_ZRODEL>` ze starych plików źródłowych i rozpakuj tam paczkę:

```
cd <KATALOG_ZRODEL>
unzip -o ~/mini-dm-fb87c13.zip
```

> *Nigdy nie rozpakowuj paczki do `<KATALOG_DANYCH>` i nie czyść tego katalogu.*

4. Zatrzymanie aplikacji i kopia zapasowa bazy

4.1. Zachowaj obecny obraz pod osobną nazwą, żeby można było do niego wrócić. Jeśli `mini-dm:prev` został po poprzednim wdrożeniu, zostanie nadpisany, i tak ma być.

```
docker tag mini-dm:latest mini-dm:prev
```

4.2. Zatrzymaj aplikację. SQLite pozwala na jednego zapisującego, a kopia musi być spójna:

```
cd <KATALOG_COMPOSE>
docker compose stop mini-dm
```

4.3. Zrób kopię **wszystkich** plików bazy, czyli `mini-dm.db` oraz, jeśli istnieją, `mini-dm.db-wal` i `mini-dm.db-shm` :

```
STAMP=$(date +%Y%m%d-%H%M)
mkdir -p <KATALOG_KOPII>/mini-dm-$STAMP
cp -a <KATALOG_DANYCH>/mini-dm.db* <KATALOG_KOPII>/mini-dm-$STAMP/
ls -l <KATALOG_KOPII>/mini-dm-$STAMP/
```

4.4. Zanotuj stan bazy przed wdrożeniem. Polecenie uruchamia na chwilę stary obraz w trybie tylko do odczytu:

```
docker run --rm -v <KATALOG_DANYCH>:/app/data mini-dm:prev node -e '
const db = require("better-sqlite3")("/app/data/mini-dm.db", { readonly: true,
fileMustExist: true });
console.log("zamowienia:", db.prepare("SELECT COUNT(*) AS c FROM purchase_orders").get().c);
console.log("historia:  ", db.prepare("SELECT COUNT(*) AS c FROM
purchase_order_history").get().c);
console.log("migracje:  ", db.prepare("SELECT version FROM schema_migrations ORDER BY
version").all().map(r => r.version).join(","));
'
```

Zapisz liczby `zamowienia` i `historia` . Pole `migracje` powinno pokazać `1,2,3,4` .

Starsza wersja: jeśli `migracje` pokazuje `1,2,3` , serwer ma wersję sprzed migracji nr 4 (zob. uwagę w punkcie 1).

Jeśli `migracje` pokazuje już `1,2,3,4,5` , nowa wersja była już wdrożona. Wdrożenie możesz powtórzyć, migracja nr 5 nie wykona się drugi raz.

5. Budowa nowego obrazu

```
cd <KATALOG_ZRODEL>
docker build -t mini-dm:latest .
```

Budowa powinna zakończyć się bez błędów. Etap `npm run build` jest w niej wymagany.

6. Kontrola uprawnień do plików bazy

Kontener działa jako nieuprzywilejowany użytkownik `app` . Przy jednym z wcześniejszych wdrożeń pliki bazy należały do innego użytkownika z prawami `644` . Aplikacja wtedy startowała i wyświetlała dane, ale **pierwszy zapis kończył się błędem**.

```
ls -l <KATALOG_DANYCH>/
```

Jeśli pliki `mini-dm.db*` nie mają praw zapisu dla „others" (`-rw-rw-rw-`), wykonaj:

```
sudo chmod 666 <KATALOG_DANYCH>/mini-dm.db*
```

Sam katalog również musi być zapisywalny dla kontenera (wcześniej miał prawa `777`).

7. Uruchomienie nowej wersji

```
cd <KATALOG_COMPOSE>
docker compose up -d mini-dm
docker compose logs --tail 50 mini-dm
```

W logach **nie** powinno być błędów typu `Migration ...` , `SQLITE_...` ani `foreign key violation` . Migracje nie wypisują komunikatu o sukcesie, więc brak błędów i działający serwer oznaczają, że start się udał.

Migracja nr 5 działa w jednej transakcji. Jeśli się nie powiedzie, baza zostaje bez zmian, a kontener nie wystartuje (w logach będzie błąd). Przejdź wtedy do kroku 9.

8. Weryfikacja

8.1. Baza. Uruchom to samo sprawdzenie co w kroku 4.4, tym razem na nowym obrazie, z dodatkowym sprawdzeniem kolumn:

```
docker run --rm -v <KATALOG_DANYCH>:/app/data mini-dm:latest node -e '
const db = require("better-sqlite3")("/app/data/mini-dm.db", { readonly: true,
fileMustExist: true });
console.log("zamowienia:", db.prepare("SELECT COUNT(*) AS c FROM purchase_orders").get().c);
console.log("historia:  ", db.prepare("SELECT COUNT(*) AS c FROM
purchase_order_history").get().c);
console.log("migracje:  ", db.prepare("SELECT version FROM schema_migrations ORDER BY
version").all().map(r => r.version).join(","));
const cols = db.prepare("PRAGMA table_info(purchase_orders)").all().map(c => c.name);
console.log("kolumny:   ", cols.slice(cols.indexOf("warehouse_name"),
cols.indexOf("warehouse_name") + 3).join(", "));
console.log("integralnosc:", db.pragma("integrity_check", { simple: true }), "/ FK:",
db.pragma("foreign_key_check").length);
'
```

Oczekiwany wynik:

- `zamowienia` i `historia` mają **te same liczby** co w kroku 4.4,
- `migracje` pokazuje `1,2,3,4,5` ,
- `kolumny` pokazuje `warehouse_name, shipping_line, has_eur1_certificate` ,
- `integralnosc` pokazuje `ok / FK: 0` .

Starsza wersja: `migracje` zmienia się z `1,2,3` na `1,2,3,4,5` , a liczby zamówień i historii pozostają te same.

8.2. Aplikacja w przeglądarce (`https://<ADRES_DEMO>`):

1. Zaloguj się. Kafelki na **Dashboardzie** pokazują te same liczby co przed wdrożeniem.
2. Na liście **Zamówień** zamiast kolumny „Wartość" jest kolumna „Cena za kg", z ceną portową i walutą.
3. Otwórz **Nowe zamówienie** (bez zapisywania). Formularz wygląda tak jak przed wdrożeniem i **nie ma** pola „Armator".
4. **Test zapisu (obowiązkowy):** otwórz edycję dowolnego zamówienia. Obok pola „Magazyn" jest puste pole „Armator" z licznikiem `0/50` . Wpisz np. `TEST` i zapisz.
5. Ponownie otwórz edycję tego zamówienia. Pole „Armator" pokazuje `TEST` , a w historii zmian jest wpis „Armator: — → TEST".
6. Jeśli to nie jest zamówienie testowe, wyczyść pole „Armator" i zapisz, żeby przywrócić poprzedni stan. W historii pojawi się wpis „Armator: TEST → —".

Wdrożenie jest zakończone dopiero po udanym teście zapisu (punkty 4–5), a nie po samym wyświetleniu strony.

9. Wycofanie zmian (rollback)

Wykonaj te kroki tylko wtedy, gdy krok 7 lub 8 się nie powiedzie.

Zwykle wystarczy **powrót do poprzedniego obrazu**, a dane wprowadzone w międzyczasie zostają zachowane. Poprzednia wersja (`69b80f5`) działa poprawnie na bazie po migracji nr 5: nie zna kolumny `shipping_line` , więc ją pomija. Wpisane wartości „Armator" zostają w bazie i znów będą widoczne po ponownym wdrożeniu nowej wersji.

```
cd <KATALOG_COMPOSE>
docker tag mini-dm:prev mini-dm:latest
docker compose up -d mini-dm
docker compose logs --tail 50 mini-dm
```

Bazę z kopii przywracaj tylko wtedy, gdy sprawdzenie z kroku 8.1 pokazało inne liczby zamówień lub historii albo błąd integralności, albo gdy serwer miał *starszą wersję* (`migracje: 1,2,3` w kroku 4.4). Dane wprowadzone po kroku 7 zostaną wtedy utracone. Przywróć bazę **razem** z obrazem:

```
cd <KATALOG_COMPOSE>
docker compose stop mini-dm

# usuń bieżące pliki -wal/-shm, żeby nie zostały po nowej wersji
sudo rm -f <KATALOG_DANYCH>/mini-dm.db-wal <KATALOG_DANYCH>/mini-dm.db-shm
sudo cp -a <KATALOG_KOPII>/mini-dm-<STAMP>/mini-dm.db* <KATALOG_DANYCH>/
sudo chmod 666 <KATALOG_DANYCH>/mini-dm.db*

docker tag mini-dm:prev mini-dm:latest
docker compose up -d mini-dm
docker compose logs --tail 50 mini-dm
```

Po rollbacku powtórz sprawdzenie z kroku 4.4 na obrazie `mini-dm:latest` . Liczby i lista migracji powinny być takie same jak przed wdrożeniem.

10. Po udanym wdrożeniu

- Zostaw kopię z kroku 4.3 i obraz `mini-dm:prev` przez kilka dni. Gdy klient potwierdzi, że wszystko działa, możesz usunąć obraz poleceniem `docker image rm mini-dm:prev` .

- Usuń paczkę `mini-dm-fb87c13.zip` z serwera.